

Neural Networks Lecture 1 – Extra Content

Data Science Toolbox Week 7

Discussion Assignment

We know that gradient descent let's us work out what direction to move in to decrease loss and improve performance. Explain backpropagation and its role in updating parameters through deep MLPs.

The next slides is some extra material on gradient descent and back prop with some suggested reading

Gradient descent

- Differentiate our loss function with respect to our model weights:

$$\nabla L_W = \begin{pmatrix} \partial_{W_{11}} L \\ \vdots \\ \partial_{W_{nM}} L \end{pmatrix}$$

- Gradient tells us which way is downhill on our loss landscape. We take a small step in that direction:

$$W_{ij} \leftarrow W_{ij} - LR \frac{\partial L}{\partial W_{ij}}$$

- Where LR = Learning Rate, positive real hyperparameter which affects network convergence.

Learning = minimizing a loss function with respect to your network parameters

Backpropagation – how to calculate the gradient?

We consider the contribution of a single training example to the loss function:

Loss function with respect to expected labels and activations of the last (L -th) hidden layer of neurons

$$L_0 = \sum_{j=0}^{n_L-1} (a_j^{(L)} - y_j)^2$$

$$a_j^{(L)} = \sigma(z_j^{(L)})$$

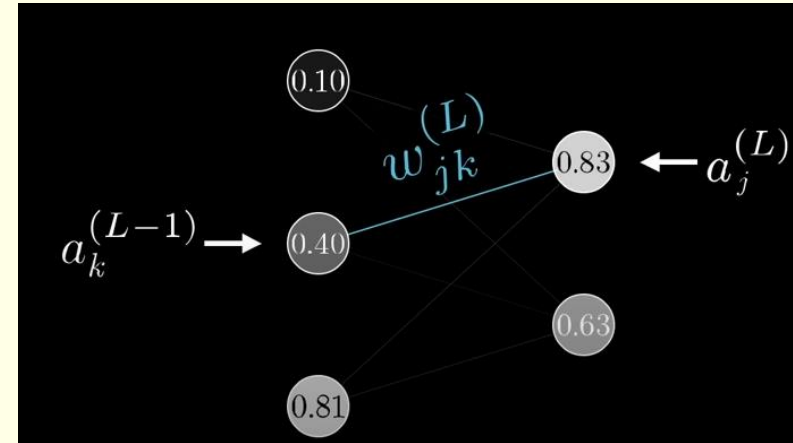
$$z_j^{(L)} = b_j^{(L)} + \sum_{k=0}^{n_{L-1}-1} w_{jk}^{(L)} a_k^{(L-1)}$$

Chain Rule for a weight $w_{jk}^{(L)}$:

$$\frac{\partial L_0}{\partial w_{jk}^{(L)}} = \sum_{j=0}^{n_L-1} \frac{\partial z_j^{(L)}}{\partial w_{jk}^{(L)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \frac{\partial L_0}{\partial a_j^{(L)}}$$

Chain Rule for activation $a_k^{(L-1)}$:

$$\frac{\partial L_0}{\partial a_k^{(L-1)}} = \sum_{j=0}^{n_L-1} \frac{\partial z_j^{(L)}}{\partial a_k^{(L-1)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \frac{\partial L_0}{\partial a_j^{(L)}}$$



Exercise: calculate the partial derivatives for the weights (consider a ReLU activation function).

Can you do the same for the corresponding bias chain rule for $b_j^{(L)}$?

What might go wrong with this training paradigm?

1. How do we deal with such a large amount of training data (rather than looking at examples one at a time)?
2. Under what circumstances would gradient descent achieve a global minimum? Where might it get stuck at a local minimum?

SGD, Momentum and Adam:

- Stochastic Gradient Descent (SGD): calculates gradient on a few training examples per update which helps explore loss surfaces and is more memory efficient
- We can also add Momentum (an exponentially weighted average of past gradients, i.e. the first moment) which smooths updates so that you don't bounce around the loss surface as much.
- We can also add Adaptive Learning Rates (an exponentially weighted average of squared gradients, i.e. the second moment) which scales each parameter's step size based on how volatile its gradients are.
- Adam is the name of the optimizer that combines SGD, momentum and adaptive learning rates.

$$g_t = \nabla_{\theta} L_t \quad (\text{current gradient})$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{1st moment - mean})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{2nd moment - variance})$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (\text{bias correction})$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{bias correction})$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (\text{gradient step})$$

α = Learning rate (default 0.001)

β_1 = Moment decay (default 0.9)

β_2 = RMS decay (default 0.999)

ϵ = Numerical stability (default 10^{-8})

Further resources

- If you don't read AI papers you'll never learn how to read AI papers, so go and read some of the ones from the slides but also...
- Go and watch the first four videos of 3Blue1Brown's videos on neural networks (especially the videos on backpropagation) : https://youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi&si=gY0nycdjWu2kf80g
- Bonus, watch Welch labs video series on How models learn: <https://youtu.be/NrO20Jb-hy0?si=SCTNtHt2DR4ZvVCu>
- The StatQuest Illustrated Guide to Neural Networks and AI (also the StatsQuest YouTube channel: <https://statquest.org/>)